

Vie Project - Chatbot Training Document

1) Project Overview

Vie is a multi-tenant web platform for organizations to manage operations, processes, issues, audits, and controlled documents.

The application is built with Next.js App Router and TypeScript, with one central (master) database plus one tenant database per organization.

Primary goals of the product:

- onboard companies and create isolated organization workspaces
- manage sites, processes, and role-based teams
- run issue management workflows (including review and verification)
- execute audit workflows from planning to findings submission
- control document lifecycle and documentary evidence workflows

2) Technology Stack

- Frontend/App: Next.js (App Router), React, TypeScript, Tailwind CSS, Radix UI
- Backend: Next.js API routes
- Authentication: NextAuth (credentials + OAuth providers)
- Main DB: PostgreSQL with Prisma ORM
- Tenant DBs: PostgreSQL via raw SQL and pooled `pg` connections
- Storage: AWS S3 (documents, audit files, rich text uploads)
- Email: SMTP via Nodemailer
- Validation and forms: Zod + React Hook Form

3) Core Architecture

3.1 Two-Database Multi-Tenant Model

1. Main database (shared across all organizations):

- users, auth sessions/accounts
- organizations and user membership
- invitation metadata
- tenant database connection records

2. One tenant database per organization:

- operational organization data (sites, processes, issues, audits, documents, etc.)
- role/assignment mappings and tenant-specific records

3.2 Tenant Provisioning Flow

When a new organization is created:

1. create master organization record
2. create tenant database (for example `org\_{orgId}`)
3. run tenant SQL migrations
4. seed tenant data from onboarding payload
5. link creator as organization owner/member

3.3 Tenant Resolution at Runtime

- Organization context is resolved from route path and/or host (subdomain mode).
- API routes validate membership and enforce tenant isolation.
- Tenant queries use pooled connections keyed by `orgId`.

4) Authentication and Authorization

4.1 Authentication

- Credentials login (email/password + email verification checks)
- OAuth login (Google, GitHub, Apple, Atlassian)
- NextAuth session-based auth with shared-cookie support for subdomains

4.2 Authorization (RBAC)

- Role-based permissions by organization (owner/admin/manager/member + additional roles)
- Permission checks applied in APIs and UI access gates
- Invite flow supports:
 - existing logged-in user acceptance
 - new user acceptance with password setup

5) Main User Journeys

5.1 Organization Onboarding

1. user signs up or logs in
2. user creates organization through multi-step setup

3. backend provisions tenant DB and seeds initial data
4. user is redirected into org dashboard

5.2 Process and Issue Management

1. user selects site and process context
2. user creates/issues and tracks status via board/backlog views
3. privileged roles review/verify and move items through workflow

5.3 Audit Workflow (High Level)

1. step 1: define audit program
2. step 2: generate audit plan and assign auditors
3. step 3: fill checklist findings and submit to auditee
4. status transitions are stored in audit tables and shown in audit list

5.4 Document Workflow

1. create controlled document
2. review and approval workflow
3. revision history and obsolete/retention handling
4. documentary evidence capture and verification flows

6) Functional Domains for Training

Use these domains as chatbot topic clusters:

1. Authentication and account management
2. Organization setup and tenant provisioning
3. Dashboard navigation and org context
4. Sites and processes
5. Issues lifecycle (create, review, verify, board states)
6. Audit programs/plans/checklists/findings
7. Documents lifecycle and evidence records
8. Roles, permissions, teams, and settings
9. File upload/download behavior and S3 usage
10. Subdomain routing and tenant URL behavior

7) Important Route Families

Public/auth routes:

- `/`
- `/auth`
- `/auth/invite`
- `/auth/resolve`
- `/organization-setup/step1` ... `/organization-setup/step11`

Org dashboard routes:

- `/dashboard/[orgId]`
- `/dashboard/[orgId]/processes`
- `/dashboard/[orgId]/issues/\*`
- `/dashboard/[orgId]/audit/\*`
- `/dashboard/[orgId]/documents/\*`
- `/dashboard/[orgId]/settings/\*`

API route families:

- `/api/auth/\*`
- `/api/organization/create`
- `/api/organization/list`
- `/api/organization/[orgId]/\*` (members, roles, permissions, sites, processes, issues, audit, documents, notifications, etc.)
- `/api/files/\*`

8) Key Data Concepts

Master DB entities:

- User, Account, Session
- Organization, UserOrganization
- OrgDatabaseInstance
- Invitation, VerificationToken

Tenant DB entity groups:

- organization info, sites, processes
- users-to-site/process mappings
- issues, sprints, activity logs
- audit programs/plans/assignments/findings
- document records/history/obsolete lifecycle
- documentary evidence records

9) Chatbot Behavior Guidance (Recommended)

Use this guidance when training your chatbot:

1. Always identify organization context first (`orgId` or slug).
2. Distinguish auth failure from permission failure:
 - 401 = not authenticated
 - 403 = authenticated but not allowed
3. Explain role-based restrictions clearly before suggesting actions.
4. For troubleshooting, ask for:
 - current route URL
 - user role
 - organization slug or id
 - exact error message
5. For data questions, mention whether data lives in:
 - main DB (shared metadata/auth)
 - tenant DB (organization operational data)
6. Never expose secrets or raw credentials in responses.

10) Security and Compliance Notes

- Do not include real `.env` values in chatbot training data.
- Rotate any leaked credentials before sharing data externally.
- Keep training data sanitized:
 - remove passwords, tokens, API keys, SMTP credentials, and cloud secrets
 - replace with placeholders like ``

11) Example Intents and Answers for Bot Training

Intent: "How is a new organization created?"

Expected answer:

"The app creates an organization in the main DB, provisions a dedicated tenant database, runs tenant migrations, seeds onboarding data, and links the creator as owner."

Intent: "Why can a user see 403 on org API?"

Expected answer:

"403 means the user is logged in but not authorized for that organization or action. Verify membership, role permissions, and resolved org context."

Intent: "Where are audit findings saved?"

Expected answer:

"Audit findings are stored in tenant audit tables linked to the generated audit plan. Submission updates plan status and timestamps."

Intent: "How does subdomain routing work?"

Expected answer:

"Tenant subdomains are internally rewritten to dashboard org routes, while auth/api/static paths are excluded from rewrite logic."

12) Suggested Training Chunks

When feeding this document to a chatbot, chunk by these sections:

1. Overview + architecture
2. Auth + RBAC
3. Route map + API families
4. Domain workflows (issues/audits/documents)
5. Security and troubleshooting guidance

This chunking improves retrieval quality for RAG systems.

13) Source Reference Files

Primary documentation and code references used to build this training document:

- `docs/ARCHITECTURE.md`
- `docs/SUBDOMAIN-MULTITENANCY.md`
- `docs/AUDIT\_FLOW\_STEPS\_1\_2\_3.md`
- `prisma/schema.prisma`
- `src/lib/auth.ts`
- `src/lib/request-context.ts`
- `src/lib/db/tenant-pool.ts`
- `src/app/api/organization/create/route.ts`
- `src/components/dashboard/Sidebar.tsx`
- `src/app/dashboard/[orgId]/audit/create/1/page.tsx`

14) PDF Export Note

This markdown file is intentionally structured for clean export to PDF.

If your environment has a PDF converter (Pandoc, VS Code Markdown PDF extension, or browser print-to-PDF), export this file as:

``docs/Vie-Chatbot-Training-Document.pdf``